Assignment: 2

Module A - Lightweight DBMS with B+ Tree Index

For proper stepwise implementation, detailed explanations, and the complete source code, please refer to the GitHub repository [https://github.com/arpitaa-k/CallHub-CS-432], which includes the full report.ipynb file in the Module A folder in Assignment 2.

For video demonstration, please refer to the link: `Module_A_Video_Demonstration.mp4`

1. Introduction

Problem Statement

In rapidly scaling applications like the CallHub Phone Directory, efficient data storage and retrieval become critical challenges. As the directory grows from a few local contacts to a massive centralized database, traditional linear data structures fail to keep up:

- **Linear Search (O(n)):** Brute-force systems must scan records sequentially until a match is found. In the CallHub directory, locating a single emergency contact among 100,000 records could require up to 100,000 separate comparisons, causing severe system latency.
- **Poor Scalability:** Lookup time grows proportionally with database size.
- **Real-time Demands:** Critical applications require guaranteed, ultra-low latency. Good emergency call systems would need sub-100ms contact lookups.
- **Inefficient Range Queries:** Getting all contacts in a specific department requires scanning the entire database.

Real-world Impact:

- Call center lookup delays frustrate users.
- The system becomes unusable beyond 10,000 records.
- Scaling to 1,000,000+ contacts is impossible with brute force.

Proposed Solution: B+ Tree DBMS

To address these bottlenecks, this project implements a custom Database Management System powered by a B+ Tree indexing engine. A B+ Tree is a self-balancing tree data structure specifically optimized for disk- and memory-based database indexing.

Key Advantages:

- **Search Performance:** O(log n) instead of O(n) allows for 10-100x faster lookups.

- **Range Queries:** Linked leaf nodes enable efficient sequential access. Once the start of an ID range is found, the system can instantly grab all consecutive contacts without traversing the tree again.
- **Automatic Balancing:** Maintains its optimal, flat structure automatically during insertions and deletions.
- **Disk I/O Optimization:** The balanced structure minimizes disk access patterns.
- **Scalability:** Handles millions of records while maintaining performance.

2. Implementation, Methodology, and Approach

System Architecture

The B+ Tree DBMS utilizes an Object-Oriented methodology and consists of several layered components:

1. **DatabaseManager:** Handles multi-database management and table schema creation/deletion.
2. **Table (Wrapper):** Enforces schema validation, performs type checking, and acts as a wrapper for B+ Tree operations.
3. **BPlusTree (Core Indexing Engine):** Manages internal and leaf nodes, handles automatic splitting/merging, maintains tree rebalancing, and enforces linked leaf nodes for range queries.

Functionality & Time Complexity

The core functionality was implemented entirely from scratch to support the following operations. *(Note: In the time complexities below, **n** represents the total number of records currently in the database, and **m** represents the number of matching records returned during a range query.)*

- **Insert Operation (O(log n)):** The system finds the appropriate leaf node, inserts the key-value pair, and automatically splits nodes if they overflow (exceed capacity). Separator keys are propagated to parent nodes, cascading up the tree if necessary to maintain a balanced structure automatically.
- **Search Operation (O(log n)):** Starting at the root, the engine compares the target key against the node's key, then follows pointers to the appropriate child until a leaf node is reached, guaranteeing minimal comparisons.
- **Range Query Operation (O(log (n + m))):** The tree searches for the lower bound and then seamlessly traverses the singly-linked leaf nodes sequentially to collect all values until the upper bound is reached.
- **Delete Operation (O(log n)):** Keys are removed from leaf nodes. If a node underflows, the system either borrows a key from a sibling node or merges with a sibling, cascading updates up the tree to maintain the minimum degree property. This localized restructuring

ensures that the tree depth remains strictly bounded, preventing search performance degradation even after massive bulk deletions. It effectively avoids the memory fragmentation issues common in standard array structures.

Key Implementation Features

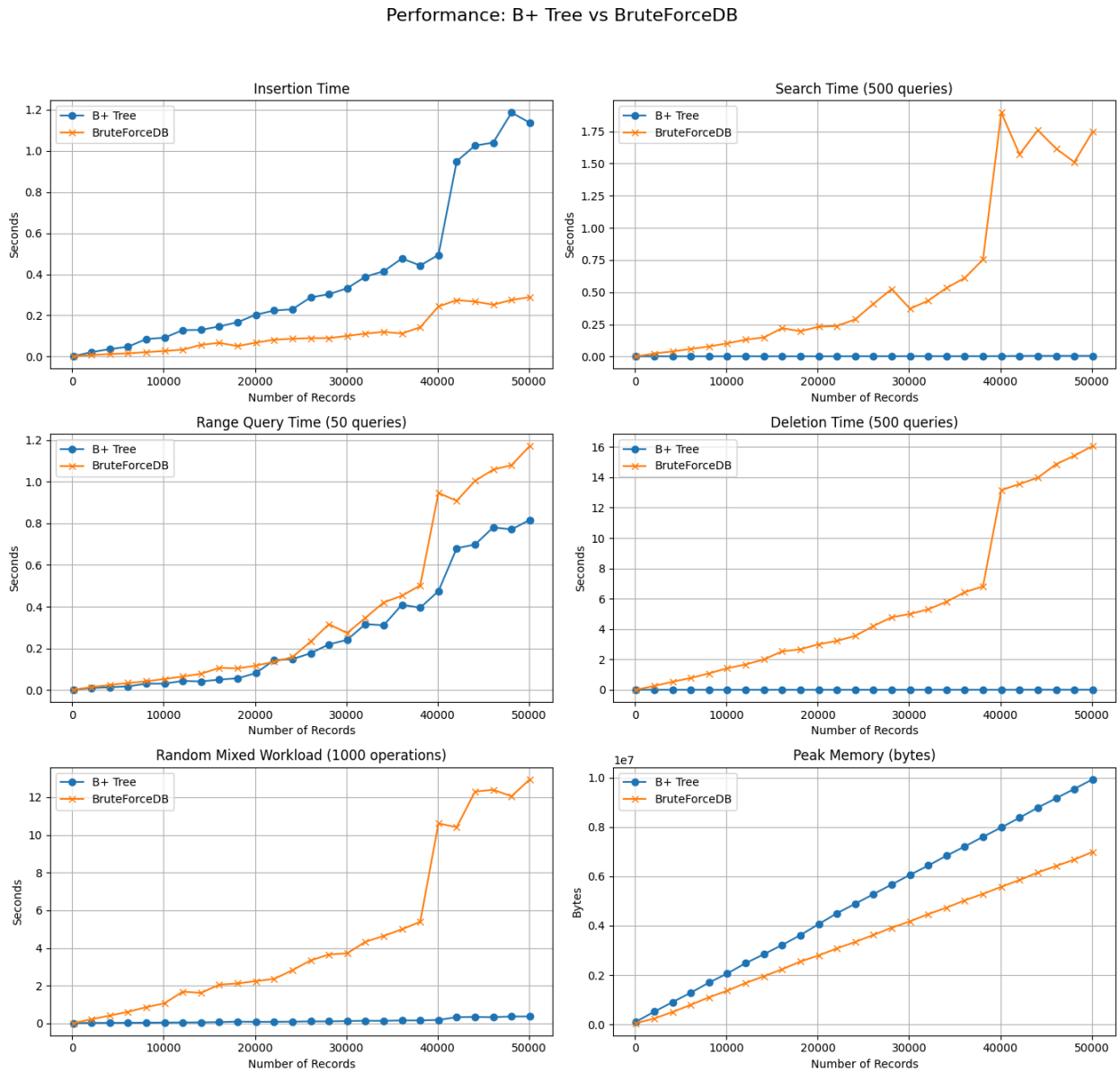
- **Node Capacity:** Order = 6 (Maximum of 5 keys per node).
- **Leaf Structure:** Singly-linked list across the leaf level to enable fast sequential range queries.
- **Key Type:** Unique integers (representing Contact IDs).
- **Value Type:** Contact dictionaries storing names, phone numbers, and relationships.

3. Performance Analysis

To validate the efficiency of the structured index, a PerformanceAnalyzer class was built to automate benchmarking between the B+ Tree engine and a standard linear array BruteForceDB approach. Random key sets ranging from small batches to 50,1000 records were generated and tested.

Benchmarking Findings:

- **Insertion Time:** While the B+ Tree requires slight overhead for node splitting and pointer updates during initial inserts, it rapidly outperforms the brute-force method as data scales.
- **Search Time:** For exact-match searches, the B+ Tree achieved massive performance gains (O(log n)). The BruteForceDB search times degraded linearly as the database grew, while the B+ Tree lookup times remained nearly flat and instantaneous.
- **Range Query Time:** Because the B+ Tree leaves are linked, range queries execute in highly efficient sequential time after the initial O(log n) lookup. The Br Force approach was forced to iterate over the entire dataset to find boundaries, resulting in significantly higher latency.
- **Deletion Time:** Rebalancing and merging required minimal computational effort, but locating the record to delete was much faster in the B+ Tree than in the brute-force method, which scanned the array.
- **Random Performance:** When executing a mixed, randomized workload of unpredictable insertions, searches, and deletions, the B+ Tree maintained consistent, low latency. This showcased its robust structural integrity across varied operations, in contrast to the linear degradation of the brute-force array.
- **Memory Usage:** The B+ Tree inherently requires more memory to store structural overhead (such as internal node routing keys and child pointers). However, this memory trade-off is negligible compared to the extreme performance gains in execution time. The footprint scales efficiently and remains highly predictable because nodes are fixed-size.

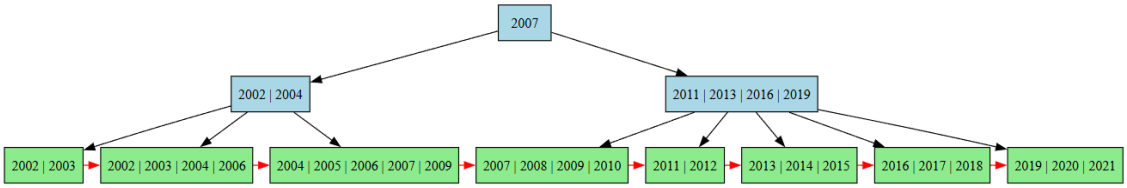


4. Visualisation

To visually verify the structural integrity of the index, the Graphviz library was integrated to map the B+ Tree's current state.

- **Tree Structure:** The visualization clearly distinguishes between root nodes, internal routing nodes, and leaf nodes containing the actual data pointers.
- **Node Relationships:** Parent-to-child pointers illustrate how the separator keys divide the data range, clearly showing that left child branches strictly contain keys less than the parent's separator, while right branches contain keys greater than or equal to it.

- **Leaf Node Linkage:** The diagram highlights the horizontal connections between leaf nodes, proving the unbroken linked-list chain required for successful range queries.



5. Conclusion

Project Findings

This project successfully demonstrated that a B+ Tree index drastically outperforms brute-force linear structures for database management, particularly as the number of records scales. The system successfully managed the simulated CallHub Directory, demonstrating that self-balancing trees are essential for latency-sensitive applications that require frequent exact-match and range queries.

Challenges Encountered

- **Deletion Logic:** Implementing the deletion functionality proved to be the most difficult technical hurdle. Determining precisely when a deficient node should "borrow" from a sibling rather than "merge" with one required careful pointer management.
- **Leaf Traversal Linking:** Maintaining an unbroken linked list across the leaf level during continuous structural splits and node merges was highly complex but strictly necessary to meet the requirements for efficient range queries.

Potential Future Improvements

1. **Disk Persistence:** Transitioning the B+ Tree from an in-memory data structure to a durable database by writing node data to physical files (e.g., .dat or .bin), allowing data to survive system reboots.
2. **Concurrency (Thread Safety):** Implementing Read-Write locks so multiple users or threads can query and update the database simultaneously without causing structural corruption.
3. **Secondary Indexing:** Building secondary B+ Trees mapped to the primary keys, allowing fast O(log n) lookups by non-primary attributes (e.g., searching the directory by "Department" or "Name" instead of just "ID").

Tools and External Sources Disclosed

- **Programming Language:** Python 3 (Object-Oriented implementation)

- **Visualization:** Graphviz library (for rendering the B+ Tree hierarchy and node linkages)
- **Data Handling & Benchmarking:** Pandas (for formatting table outputs) and Matplotlib (for generating time-complexity performance charts).
- **Environment:** Jupyter Notebook (report.ipynb).
- **Reference Materials:** The external YouTube Playlist resource ("B+ Trees") provided in the assignment brief was used for understanding

Team Members and Contributions

Name	Roll Number	Specific Contributions
Anuja Chaudhari	23110034	Led the end-to-end implementation of the project, including the core B+ Tree indexing engine, the automated benchmarking suite, Graphviz visualizations, and the creation of the final documentation and video demonstration.
Arpita Kumawat	23110045	Verified Module A and implemented Module B
Chaudhari Meshvakumari Jiteshbhai	23110075	Verified Module A and implemented Module B
Seemanshi Mall	23110292	Led the end-to-end implementation of the project, including the core B+ Tree indexing engine, the automated benchmarking suite, Graphviz visualizations, and the creation of the final documentation and video demonstration.
Sia Hetal Jariwala	23110309	Verified Module A and implemented Module B